

Technical Evaluation Report

JAVA Assessment

Candidate:	nitin.bharti@irissoftware.com
Email:	nitin.bharti@irissoftware.com
Tech Stack:	JAVA
Completed:	August 18, 2026 at 03:50 PM
Report Generated:	August 18, 2026 at 03:51 PM

Overall Assessment

73.7 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	21.0 minutes
Planned Duration	65 minutes
Time Efficiency	32.3% of planned time ✓

Category Scores

Category	Score
Kafka	73.7/100

Communication Assessment

Metric	Score
--------	-------

Confidence Level	7.0/10
English Proficiency	7.2/10
Clarity Score	6.5/10

Strengths

- Correctly distinguished same consumer group behavior from different consumer groups
- Recognizes Kafka is more useful in larger or more complex systems than trivial point-to-point communication
- Correctly connected same-key partitioning with preserving order for related events
- Correctly distinguished ordering guarantees within a partition versus across partitions
- Core Kafka consumer group semantics were correct

Areas for Improvement

- Be more accurate with technology names (e.g. Springwood vs Spring Boot)
- Explain the main use cases more concretely: microservice decoupling, event-driven architecture, log aggregation, stream processing, and high-throughput data pipelines.
- Use cleaner partition examples to avoid confusion in numbering and assignment.
- State the key advantages explicitly: scalability, durability, fault tolerance, high throughput, and independent consumers.
- Explicitly state that ordering within a partition is maintained by Kafka assigning monotonically increasing offsets as records are appended to that partition

Detailed Question Feedback

Q1: Please explain your most recent project in detail. Describe the technologies used, your role and responsibilities, the challenges you faced, and how you overcame them. Also explain the architecture and any significant technical decisions you made.

Category: Project Experience | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

I was part of the Citibank project under which in my recent assignment so I was worked for the Banamix diversity project right so the main focus of that project is the one of the retail unit which is Banamix is basically moving out from the bank and so we have to provide all the standalone I mean the functionality so that once that unit is basically moved out from the system right it will work as is so this is the project detail on very high level in terms of functionality so inside that we are using the spring springwood Java 17 and along with that we are having the microservices and autosys and gel files we are using to run the jobs shell script are basically we're using to trigger the jobs right and apart from that we're using light speed and hardness for build and deployment as a tool may but we're using and get we are using for code worsening so these are the technical skill set my role was basically I was as a team leader to make sure on daily basis it takes from call right so we have to have we just basically had a scrum call daily in the morning and we have to just I need to check what all things we as a team have done yesterday what is the plan for today's for the current sprint assignment and if any blocker is there so if any blocker over the call I mean we usually try to fix that and so that we can proceed with the assignment task within a timeline for which is three weeks for a

single sprint there are multiple challenges that we have faced as I started in a very initial phase so this so the maintaining the code the naming convention and the different different standards that we have to follow right the pull request challenges so all those things I as a team sorted out and and there are some other challenges inside that also like in the project so we are having notification need to be enabled so we prepare that notifications screen and we provide the backend logic to fetch the notification save the notification having different checks and validations so all those things are basically we have done using the Java code to overcome that one of the challenge that I would highlight here is as a business they are faced sometimes if some backend services not running so business would not be aware what is actual issue and for that we will come basically come up with an idea to prepare one page within the portal so that they would be able to see which services basically up and running and which is down if something is not working properly so I mean they know right what exactly the issue is and they can reach out to the same spark of that particular service which is not up and running and that will be fixed as compared to the earlier issue as I mean we were the business itself is not aware what is the issue so as a tech team they have to analyze the entire issue in the entire services and they have to find out the root cause and then they have to fix that so this is the challenge that we have overcome technical decision I mean we have added as I told prepare that the portal right so that is basically we have established using using this actuator spring actuator basically so we use that functionality to organize the combat challenge and technically we have fixed that

Good: Correctly identified technologies used in the project, Recognized significant technical decision made during the project

Improve: Provide clear explanations of challenges faced and how they were overcome, Be more accurate with technology names (e.g. Springwood vs Spring Boot), Offer more context for significant technical decisions

Detailed Feedback:

The candidate's answer provides a good overview of their recent project, but lacks depth and technical accuracy in some areas. They correctly identified the technologies used (Spring, Java 17, microservices, autosys, etc.) and their role as team leader. However, they incorrectly stated that Springwood is a technology used in the project (it's likely a typo for Spring Boot). The candidate also failed to provide clear explanations of the challenges faced and how they were overcome, instead providing vague statements about 'pull request challenges' and 'notification need to be enabled'. They did correctly identify one significant technical decision made during the project, which was to use Spring Actuator to create a portal for monitoring backend services. However, their explanation of this decision is brief and lacks context.

Q2: What is Kafka and what is the use case? Advantage of it and when to implement it.

Category: Kafka | Question Category: FSUpskilling | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

So Kafka is basically an event-driven messaging system and that is basically asynchronous which is the main feature of it. So if you talk about the Kafka, so it established a communication channel between two systems and so that they can communicate in an asynchronous manner. So if system A push something or publish something onto the Kafka channel, system A would complete that work once the data is published and it's a responsibility to the consumer who is basically continuously listening to that queue, sorry, the channel and they basically process the data and if they can also notify it back to the Kafka channel and to the producer itself. The use case, so Kafka is basically used to communicate as I told the channel between two more than two systems. So it is basically asynchronous way of establishing a connection. So suppose I'm having payment as a service and order a service. So suppose as a developer, I complete the payment, I'm basically initiating the payment. Now once the validations and whatever things that need to be done at the order service side is being done, so it submit the that or publish the data onto the Kafka to proceed with the payment things. So once the order service submit the data or publish the data, so it send to the Kafka queue and from there, I mean before that, so once it is published, so the payment status at the order service is basically the pending payment pending. Now it is being done from the order side. Now it's responsibility of the payment service to process the data and validate the card, the balance, the eligibility of the card, the experience, all those things and once it is being done, they need to update the same in the Kafka queue and they need to update the same in the Kafka queue and then it is being notified to the order service again that payment is successful and the same has been updated in the database also with the updated status whether it's payment is passed or I mean the success or fail accordingly. The advantage is basically because this is asynchronous nature, so it is being highly recommended when we are having, we want a system to be work on asynchronous mode and when we are having a complex system, like we are having a complex source and consumer, I mean the source, my mean source, I need producer and we need some channel to communicate or broadcast the data right and so in that case, it is good to choose with the Kafka system and if we are having less complexity systems like one or two, so it would not be recommended only in the case of complexity, the Kafka would work on very good manner.

Score: 63/100

Good: Correctly identifies asynchronous communication as a core use case, Demonstrates basic understanding of producer-consumer flow, Provides a relevant real-world example instead of only theory, Recognizes Kafka is more useful in larger or more complex systems than trivial point-to-point communication

Improve: Define Kafka more precisely: it is a distributed event streaming platform/message broker built around topics, not just a generic 'channel.', Explain the main use cases more concretely: microservice decoupling, event-driven architecture, log aggregation, stream processing, and high-throughput data pipelines., State the key advantages explicitly: scalability, durability, fault tolerance, high throughput, and independent consumers., Clarify 'when to implement it': use Kafka when systems need asynchronous decoupled communication, replayable events, or high-volume streaming; avoid implying it is only for 'complexity' in a vague sense., Avoid the misleading implication that Kafka directly notifies the producer back; instead, say another service can consume an event and publish a follow-up event to another topic.

Detailed Feedback:

The candidate shows a basic practical understanding of Kafka as an asynchronous, event-driven messaging mechanism used for communication between systems. They correctly describe the producer/consumer pattern at a high level and give a relevant microservices example involving order and payment services. They also answer the 'use case' and 'when to implement it' parts in a basic way by saying Kafka is useful in more complex distributed systems where asynchronous communication is needed. However, the answer is only average because it stays quite generic and misses several important Kafka-specific points that the question asked for. Kafka is not just a generic 'channel' or 'queue'; more accurately, it is a distributed event streaming platform / distributed publish-subscribe system built around topics, partitions, brokers, and durable message storage. The candidate repeatedly calls it a 'channel' and once corrects themselves from 'queue', which suggests partial but not strong conceptual precision. Their explanation of the payment flow is understandable, but the statement that the consumer updates Kafka and then Kafka notifies the producer back is oversimplified and somewhat misleading. In Kafka, a consumer can publish a new event to another topic, but Kafka itself does not inherently act like a direct callback notifier to the original producer. On advantages, they mention asynchronous processing and suitability for complex systems, which is valid. But they miss major advantages that are central to Kafka: high throughput, scalability via partitions, durability/persistence, fault tolerance/replication, decoupling of services, and support for multiple consumers reading the same event stream independently. On 'when to implement it,' they correctly say it is useful for complex systems and asynchronous workflows, but they do not clearly explain concrete scenarios such as event-driven microservices, log/event pipelines, analytics, audit/event sourcing, or buffering high-volume traffic. Overall, this is not a weak answer because there is real understanding and a relevant example, but it lacks precision and completeness for a Kafka question in a Java/backend context.

Q3: A Kafka topic contains 3 partitions, and a consumer group consists of 5 consumers. Explain how Kafka distributes the partitions among the consumers. Can multiple consumers in the same consumer group read from the same partition simultaneously? What happens to the extra consumers when there are more consumers than partitions?

Category: Kafka | Question Category: FSUpskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

Kafka assigned topic partition to consumer in a consumer group such that a single partition is assigned to at most one consumer in the same group. For example, if you go by the example, the current scenario, so each of the three partition will be assigned to a one distinct consumer, for example, partition is zero. The consumer consumer a basically is assigned partition one consumer B will be assigned a partition to consumer C will be assigned like that. So, this is how the assignment of the consumer and the partition will be done. Can multiply consumer in the same group read from the same partition simultaneously? No, within the same consumer group, Kafka guarantees ordering within a partition by ensuring only one consumer in the group consume from a partition at any time. So, if you want to make a, I mean, the continuation of the data, right, we need to pass the key so that using a hashing technique or whatever data we're passing, it will be decided in the same partition and it will be read accordingly and the consumer from different consumer group can read data or can read data from the same partition concurrently. This is how the Kafka behaves. So, the next question what he has been asked here is what happens to the extra consumer when the there are more consumer than the partition group? So, in that case, if we're having more consumer than the partition group within a group, so that extra consumer will be set idle and it will not be doing anything productive in that case. So, that is not the ideal to have the more number of consumer than the partition count. So, always the consumer should be maximum, I mean, it's less than or equal to the number of the partitions we have within a topic. So, this is how it should work. So, that ideal consumer act as a hot stand by instance, if any active consumers cash or leave the group Kafka trigger a rebalance and assign that freed a partition to the one of the

ideal consumers. So, in that case, if we're having ideal one, so it will be used as a backup and I mean, it will be once the one of the consumer is being down that the ideal one will be used to process the messages or consume the messages.

Score: 84/100

Good: Answered all parts of the question, Core Kafka consumer group semantics were correct, Correctly distinguished same consumer group behavior from different consumer groups, Correctly explained the effect of having more consumers than partitions

Improve: State the exact distribution more explicitly: with 3 partitions and 5 consumers, 3 consumers get one partition each and 2 consumers get no partition assignment., Avoid calling idle consumers a 'hot standby' without qualification; more accurately, they are unassigned consumers that may receive partitions after a rebalance., Use cleaner partition examples to avoid confusion in numbering and assignment.

Detailed Feedback:

The candidate gave a technically solid answer that directly addressed all three parts of the question. They correctly explained that within a consumer group, each partition is assigned to at most one consumer, so with 3 partitions and 5 consumers, only 3 consumers will actively consume and 2 consumers will remain unassigned/idle. They also correctly stated that multiple consumers in the same consumer group cannot read from the same partition simultaneously. The explanation about ordering within a partition was appropriate and accurate in this context. They additionally noted, correctly, that consumers from different consumer groups can read the same partition independently, which was extra but valid. They also correctly described that when an active consumer leaves or crashes, Kafka rebalances and one of the idle consumers may receive the freed partition. The main reason this is not scored higher is that the answer was somewhat repetitive and occasionally imprecise in wording. For example, the partition numbering/example was a little muddled, and the statement that idle consumers act as a 'hot standby' is only loosely true. They are not dedicated standby replicas in a formal sense; they are simply unassigned members of the group that may receive partitions after a rebalance. Still, the underlying meaning was clear and mostly correct. Overall, this is a good answer with correct core Kafka consumer-group behavior.

Q4: Assume a PaymentService is publishing payment events to a Kafka topic with three partitions. How does Kafka determine the target partition for each message (for example, when a message key is provided versus when it is not)? Also, is the order of messages guaranteed, and if so, how is message ordering maintained across and within partitions?

Category: Kafka | Question Category: FSUpskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

So, here in this scenario, assume a payment service is published event to a Kafka topic with the partition. How does the Kafka determine the target partition for each messages? So, there are two ways the way the Kafka is being pushing the data to the partition. So, when the producer publish the message to the Kafka. So, if producer is publishing without a key, so the data will be published to the partition on the round robin basis. Suppose the data is published, it will go first to the partition zero and the next time when the data is coming and we are not passing any key in that scenario, so the data will be published to the partition two. Likewise, in the next publish, the data will be published to the partition three. Likewise, in the round robin manner, the data will be published and it will balance the load also. I mean the load balancer way, load is also balanced in this approach. And if we are passing the key while publishing the data, so that key is basically used to publish the data to a specific partition. For example, if I am publishing now a data and in this scenario, if I am passing a key, for example, key would be any string integer or anything. And it will use more and more two algorithms to get the hash value from the particular key and according to that hash value, the data would be published in a particular partition. Likewise, for the next one, suppose in that one, I am passing the key as one, two, three, four and the data will be published to a partition zero. Next time if I want to publish the data to the next offset value, so it will be and I am passing the same key one, two, three, four. So on the next offset value, the data will be published in the same partition. It will not go to the different partition. So that will maintain the order of the data, publish order of the data whatever we are publishing by the producer. Within the partition, the order is guaranteed. Yeah, but across the partition within a topic, the order is not guaranteed.

Score: 74/100

Good: Answered both parts of the question directly, Correctly explained that keyed messages go to a deterministic partition based on hashing, Correctly connected same-key partitioning with preserving order for related events, Correctly distinguished ordering guarantees within a partition versus across partitions

Improve: Be precise with partition numbering: for three partitions, the partitions are typically 0, 1, and 2, not 3, Clarify that without a key, Kafka distributes records across partitions via the producer's partitioning strategy; calling it strict round robin is a simplification and not always exact, Explicitly state that ordering within a partition is maintained by Kafka assigning monotonically increasing offsets as records are appended to that partition

Detailed Feedback:

The candidate demonstrates solid practical understanding of Kafka partitioning and ordering, and they answered the core of what was asked. They correctly explained the two main cases: when a key is not provided, messages are distributed across partitions in a balancing manner; when a key is provided, Kafka uses the key's hash to consistently choose a partition, so the same key goes to the same partition. They also correctly stated the most important ordering rule: ordering is guaranteed within a partition, but not across partitions. That said, the answer has a few technical imprecisions. First, the example partition numbering is inconsistent with the question: the topic has three partitions, so valid partition IDs would normally be 0, 1, and 2, but the candidate said partition three at one point. That is a concrete mistake in the example. Second, saying Kafka always sends keyless messages in strict round-robin order is an oversimplification. In practice, for messages without a key, the producer/partitioner distributes records among available partitions, but the exact behavior depends on the producer partitioner and batching behavior; describing it as simple round robin is acceptable at a basic level, but not fully precise. Third, the phrase about using 'more and more two algorithms' is unclear, though the intended meaning is clearly hashing. Overall, this is a good answer because it covers the exact scope of the question: partition selection with and without keys, and message ordering within vs across partitions. It is not comprehensive enough for a very high score because of the imprecision around keyless partitioning and the incorrect partition-number example.

This report was generated by the Evalo.

© 2026 IRIS Software. All rights reserved.